Iteration E2: Handling Errors

It's apparent from the page shown in the screenshot that our application raised an exception at line 67 of the carts controller. Your line number might be different, as we have some book-related formatting stuff in our source files. If you go to that line, you'll find the following code:

```
@cart = Cart.find(params[:id])
```

If the cart can't be found, Active Record raises a `RecordNotFound` exception, which we clearly need to handle. The question arises—how?

We could silently ignore it. From a security standpoint, this is probably the best move, because it gives no information to a potential attacker. However, it also means that if we ever have a bug in our code that generates bad cart IDs, our application will appear to the outside world to be unresponsive—no one will know that an error occurred.

Instead, we'll take two actions when an exception is raised. First, we'll log the fact to an internal log file using the Rails logger facility.[49] Second, we'll redisplay the catalog page along with a short message (something along the lines of "Invalid cart") to the user, who can then continue to use our site.

Rails has a convenient way of dealing with errors and error reporting. It defines a structure called a *flash*. A flash is a bucket (actually closer to a `Hash`) in which you can store stuff as you process a request. The contents of the flash are available to the next request in this session before being deleted automatically. Typically, the flash is used to collect error messages. For example, when our `show` method detects that it was passed an invalid cart ID, it can store that error message in the flash area and redirect to the `index` action to redisplay the catalog. The view for the index action can extract the error and display it at the top of the catalog page. The flash information is accessible within the views via the `flash` accessor method.

Why can't we store the error in any old instance variable? Remember that after a